A TECHNICAL PRIMER

Model Context Protocol

MCP

Understanding how AI agents connect to tools, data, and prompts

WHAT'S INSIDE

01 What is MCP?

MCP (Model Context Protocol) is a standard protocol that allows AI models and agents to connect to external tools and data sources — in a consistent, predictable way.

```
AI Agent
   |
  MCP
   |
   +-- Calculator
   +-- Database
   +-- File System
   +-- Web Search
```

Without MCP vs. With MCP

WITHOUT MCP

Tools are defined directly inside the application:

```
@tool
def calculator(a: int, b: int):
    return a + b
```

The LangGraph / LangChain app directly knows and calls this function.

WITH MCP

A separate MCP server exposes tools instead:

```
MCP Server
|
+-- calculator
+-- weather
+-- search
+-- database
```

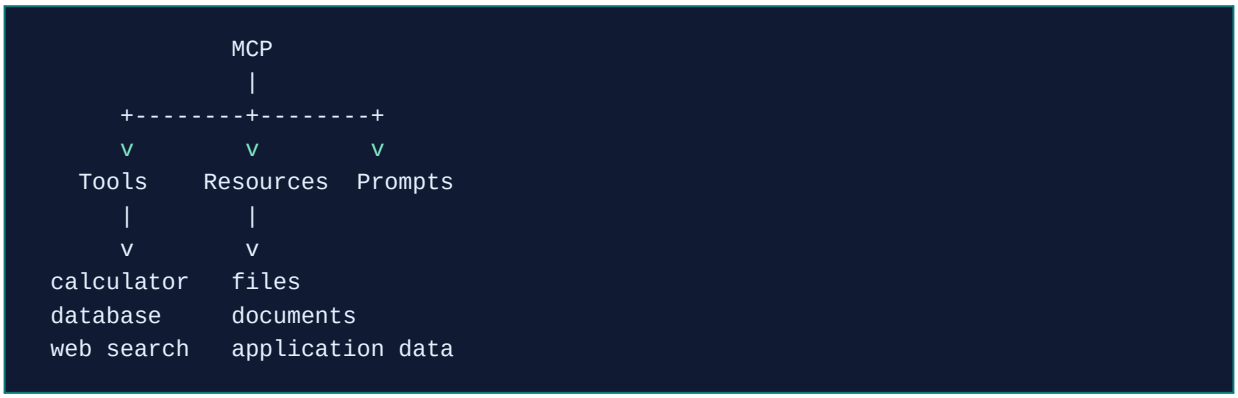
The AI application acts as an **MCP client** and connects to the MCP server.

Why MCP? — Standardization

MCP provides a common way for AI applications to connect with three kinds of building blocks:

Capability	Description	Examples
Tools	Functions the AI can execute	calculator(), database_query()
Resources	Data / context the AI can access	files, documents, application data
Prompts	Reusable prompt templates	standardized instructions for common tasks

MCP Architecture



In LangGraph

LangGraph agents connect via **MultiServerMCPClient**, and MCP servers are commonly built with **FastMCP**.

Example MCP server, built with FastMCP:

```
from fastmcp import FastMCP

mcp = FastMCP("Calculator")

@mcp.tool
def add(a: int, b: int) -> int:
    return a + b
```

The MCP server exposes the `add()` function as an MCP tool. Any other AI application can connect to the server and use it.

> **Important**
>
> MCP is **not** an AI model, and it is **not** an agent framework. MCP is a **protocol** that lets AI applications connect with tools, data, and prompts.

02 JSON-RPC & the Transport Layer

Why JSON-RPC instead of REST APIs?

MCP's data layer uses **JSON-RPC (JSON Remote Procedure Call)** rather than a typical REST API:

- It's **lightweight**
- Supports **bi-directional** communication
- It is **transport-agnostic**
- Supports **batching**
- Supports **notifications**

The Transport Layer

The transport layer is the mechanism that moves JSON-RPC messages between client and server. The right choice depends on the type of server:

Server Type	Transport	Notes
Local server	STDIO	Fast, secure, simple — in-process communication
Remote server	HTTP / SSE	Works across a network, supports streaming

STDIO — Local

- **Fast** — data passed directly between processes
- **Secure** — no open network port to attack; communication stays local
- **Simple** — every language can read/write stdin & stdout, no extra libraries

HTTP + SSE — Remote

- SSE (Server-Sent Events) is an **extension of HTTP**
- The server can send **multiple messages** over one open connection
- Data streams in **chunks** instead of one large JSON blob
- Ideal for **long-running tasks** or incremental updates

> **Message format vs. transport**
>
> **JSON-RPC** defines the message *format*. **STDIO / HTTP+SSE** are the *mechanisms* used to move those messages. Keeping these separate is key to understanding MCP.

03 The MCP Lifecycle

The MCP lifecycle describes the complete sequence of steps that govern how a Host (client) and a server establish, use, and end a connection during a session.

```
User
  |
  v
AI Application / MCP Host
  |
  v
MCP Client
  |
  v
Transport (stdio / Streamable HTTP)
  |
  v
MCP Server
  |
  v
Tools / Resources / Prompts
```

1 Initialization

The MCP client connects to the server and performs a handshake, negotiating the MCP protocol version plus client & server capabilities.

```
Client → initialize → Server
Client ← initialize response ← Server
```

2 Capability Negotiation

After initialization, both sides know exactly what the other supports — and the client can only use capabilities the server actually exposes.

```
Client capabilities         Server capabilities
  |-- sampling                |-- tools
  \-- roots                   |-- resources
                              \-- prompts
```

3 Operation / Communication

The real MCP communication begins. The client lists available tools, and the LLM may decide to call one — all over JSON-RPC messages.

```
Client → tools/list → Server
Server → available tools → Client

Client → tools/call → Server → calculator → Server → result → Client
```

4 Notifications

A notification is a message where the sender doesn't expect a response — used when something changes, like a resource being updated.

```
Server → notification → Client
```

5 Shutdown

When the client is finished, the connection/transport closes. For local STDIO servers, this generally means the server process terminates.

```
Client → close transport → Server process terminates
```

In one sentence

MCP lifecycle = **Start** → **Connect** → **Initialize** → **Negotiate capabilities** → **Communicate** → **Shutdown.**

Shutdown in STDIO

Shutdown Path	Sequence
Client-initiated	Close input stream to server → wait for exit → SIGTERM if needed → SIGKILL if still unresponsiv
Server-initiated	Close output stream to client → exit process

In practice, the client most commonly shuts down the server.

Special Cases

Pings

A lightweight request/response method used to check whether the other side (Host or Server) is still alive and the connection is responsive.

- Useful for checking the other side is up before full initialization
- Clients may send periodic pings during idle periods
- Prevents connections from being silently dropped by the OS, proxies, or firewalls

Error Handling

How the host (client) and server signal that something went wrong. MCP inherits JSON-RPC's standard error object format.

- Unsupported or mismatched protocol version
- Calling a method for a capability that wasn't negotiated
- Invalid arguments to a tool
- Internal server failure while processing a request
- Timeout issues or client-cancelled requests
- Malformed JSON-RPC messages

Timeout

Ensures requests don't hang forever.

- Protects against unresponsive or overloaded servers
- Ensures resources (memory, CPU) aren't held indefinitely
- Gives the user feedback instead of waiting forever

Progress Notification

Lets the client know a long-running request is still making progress.

- Client includes a progressToken in the request's _meta
- Server sends notification/progress updates while working

04 Connectors in Claude Desktop

A **connector** is a built-in feature that links Claude to MCP servers automatically — no manual setup or configuration required.

```
Claude Desktop
      |
      |  Connector (built-in)
      v
MCP Server  <---  wrapped & authenticated behind the scenes
      |
      +-- Notion
      +-- Google Drive
      +-- GitHub
      +-- Slack
```

Most Claude Desktop users are non-technical end users who simply want Claude to “talk” to their apps — Notion, Google Drive, GitHub, Slack, and more. They don't want to run servers, edit JSON, or worry about transports.

The Connector System wraps an MCP server behind the scenes and handles authentication via **OAuth** (sign in with Google, GitHub, etc).

> **Why it matters**
>
> The connector system keeps things **easy, safe, and consistent** — turning a technical protocol into a one-click experience for everyday users.

End of guide. This document summarizes the Model Context Protocol: its architecture, the JSON-RPC transport layer, the full connection lifecycle, and how Claude Desktop's connectors make it accessible to everyday users.